

generate_markdown_exports

scripts/ generate_markdown_exports.sh — the §11.4.65 document-twin exporter

Revision: 1 **Last modified:** 2026-09-01T16:20:00Z **Status:** active

Overview

scripts/generate_markdown_exports.sh is the project's canonical §11.4.65 exporter. For every in-scope *.md source it produces the sibling document twins:

Twin	Produced by	Present when
.html	pandoc --standalone, else python-markdown	always (one leg or the other must exist)
.pdf	weasyprint rendering the .html	the weasyprint CLI is on PATH
.docx	pandoc -f markdown -t docx (direct from the .md, BOB-011)	pandoc is on PATH

It is the script the pre-build sync invariant, scripts/compute-badges.sh, and scripts/testing/update_readme_doc_links.sh all sit downstream of.

Prerequisites

Every leg is **optional and independently degradable**, and a leg that cannot run writes **nothing** — the script never emits a blank or placeholder file (§11.4.3 honest SKIP, §11.4.6 no fabricated output).

- `pandoc` — HTML + DOCX. The highest-fidelity path.
- `weasyprint` (the **CLI**, not the Python module) — PDF.
- a Python interpreter that can import `markdown` — the HTML fallback used only when `pandoc` is absent.
- If neither `pandoc` nor a markdown-capable interpreter is found, the script **exits 1** and names both the interpreters it probed and the remedy. It does not half-run.

All three install without root:

```
uv tool install weasyprint #
    weasyprint CLI
uv pip install --python .venv/bin/python markdown # python-
    markdown fallback
# pandoc: a user-local tarball unpacked under ~/.local/opt +
    symlinked into ~/.local/bin
```

Usage

```
# Full in-scope sweep: project root *.md + docs/**/*.md + scripts/
    **.md
bash scripts/generate_markdown_exports.sh

# Explicit scope – regenerate ONLY these sources' twins
bash scripts/generate_markdown_exports.sh CLAUDE.md docs/
    USER_MANUAL.md
```

Inputs

Input	Meaning
<code>\$@</code> (optional)	Explicit <code>.md</code> paths. When present, only those files are converted and the discovery sweep is skipped entirely.
<code>\$BOBA_EXPORT_PYTHON</code> (optional)	Interpreter to use for the python-markdown leg. Absent → auto-resolved.

Outputs

.html, .pdf, and .docx siblings written next to each source .md. Counts of generated files are printed per twin type.

Exit codes

Code	Meaning
0	Completed (including “nothing was stale”).
1	No HTML converter available at all, or an explicitly-named argument was not an existing .md file.

Why the explicit-scope argument exists

The full sweep touches ~355 exports. A session that edits nine documents should resync exactly those nine twins: re-rendering 346 unrelated exports costs minutes of `weasyprint` time and needlessly churns the corpus that `scripts/pre_build/check_cm_export_charset_valid.sh` measures against its monotone ratchet baseline. Passing the changed files keeps the blast radius equal to the change.

An argument that is not an existing .md file is a **hard error**, never a silent skip — a typo’d path must not be able to masquerade as “nothing to do” (§11.4.201(6): a quiet zero from a blind instrument is indistinguishable from a clean corpus).

Idempotency and the staleness rule

Re-running is a no-op: a second run over unchanged sources rewrites nothing and leaves every twin byte-identical.

Staleness is **not** mtime alone. An .html that exists and is mtime-fresh but declares **no charset** is treated as STALE and regenerated, because such a file is a pre-BOB-169 charset-less fragment that would otherwise persist forever and keep re-baking mojibake into any PDF rendered from it. PDF staleness additionally keys on the .html (the PDF is derived from it), so healing the HTML half cannot leave a corrupt PDF behind.

Internal behaviour worth knowing

Capability probes test the artifact, not a prerequisite

The weasyprint probe runs `weasyprint --version` — the real entry point the render path uses. It previously also required `python3 -c "from weasyprint import HTML"` to succeed, which is a **prerequisite** probe, not an artifact probe: a weasyprint CLI installed into its own isolated environment (`uv tool install`, `pipx`, a `venv` on `PATH`) renders PDFs perfectly while being invisible to a system-interpreter import. On this host that made the script announce “*PDF support: not available*” and produce **zero PDFs while exiting 0** — a false negative of exactly the shape §11.4.201(11) names. Do not reintroduce an import-based probe for a CLI-invoked tool.

The python-markdown interpreter is resolved by trying the import

`PY_MD` is chosen by actually attempting `import markdown` against, in order: `$BOBA_EXPORT_PYTHON`, the repo’s `.venv/bin/python`, then `python3`. It is never assumed from a path’s existence (§11.4.6). Probing only system `python3` is insufficient here: this host’s `/usr/bin/python3` has no `pip` and is PEP-668 EXTERNALLY-MANAGED, so `markdown` cannot live there without `root` — while the repo `.venv` holds it with no privilege at all.

Encoding is pinned on both legs

The corpus carries Cyrillic (Боба) and §. The `python-markdown` leg opens and writes with an explicit `encoding='utf-8'` rather than inheriting the ambient locale, so a C/POSIX locale cannot mangle the read and leave the emitted `<meta charset="utf-8">` advertising a lie.

Verifying the output

charset declared on every generated export (ratcheted gate)

`bash scripts/pre_build/check_cm_export_charset_valid.sh`

the PDF text layer really preserves non-ASCII

`bash tests/unit/test_export_pdf_charset_integrity.sh`

DOCX shape + idempotency

`bash tests/unit/test_docx_export.sh`

When checking a heading survived into the HTML, match against a **whitespace-normalized** stream, not lines: pandoc hard-wraps its HTML at ~72 columns, so a heading routinely spans a newline and a line-oriented `grep -F` returns a confident, wrong zero (§11.4.201(7)(c)). Remember also that `&` is correctly emitted as `&`;

Related scripts

- `scripts/pre_build/check_cm_export_charset_valid.sh` — the gate over this script's output; reads a monotone-decreasing ratchet baseline.
- `scripts/regenerate-continuation-exports.sh` — deliberately narrow CONTINUATION-only regen; see its own guide for why it does not call this.
- `scripts/lib/export_staleness.sh` — the staleness oracle used by pre-build.
- `tests/unit/test_export_pdf_charset_integrity.sh` — the paired §1.1 guard.

Last verified: 2026-09-01 — full run over nine changed documents produced 27 twins; charset gate PASS (355/355 declaring a charset, baseline 0); PDF charset-integrity, DOCX, and staleness-oracle guards all PASS.